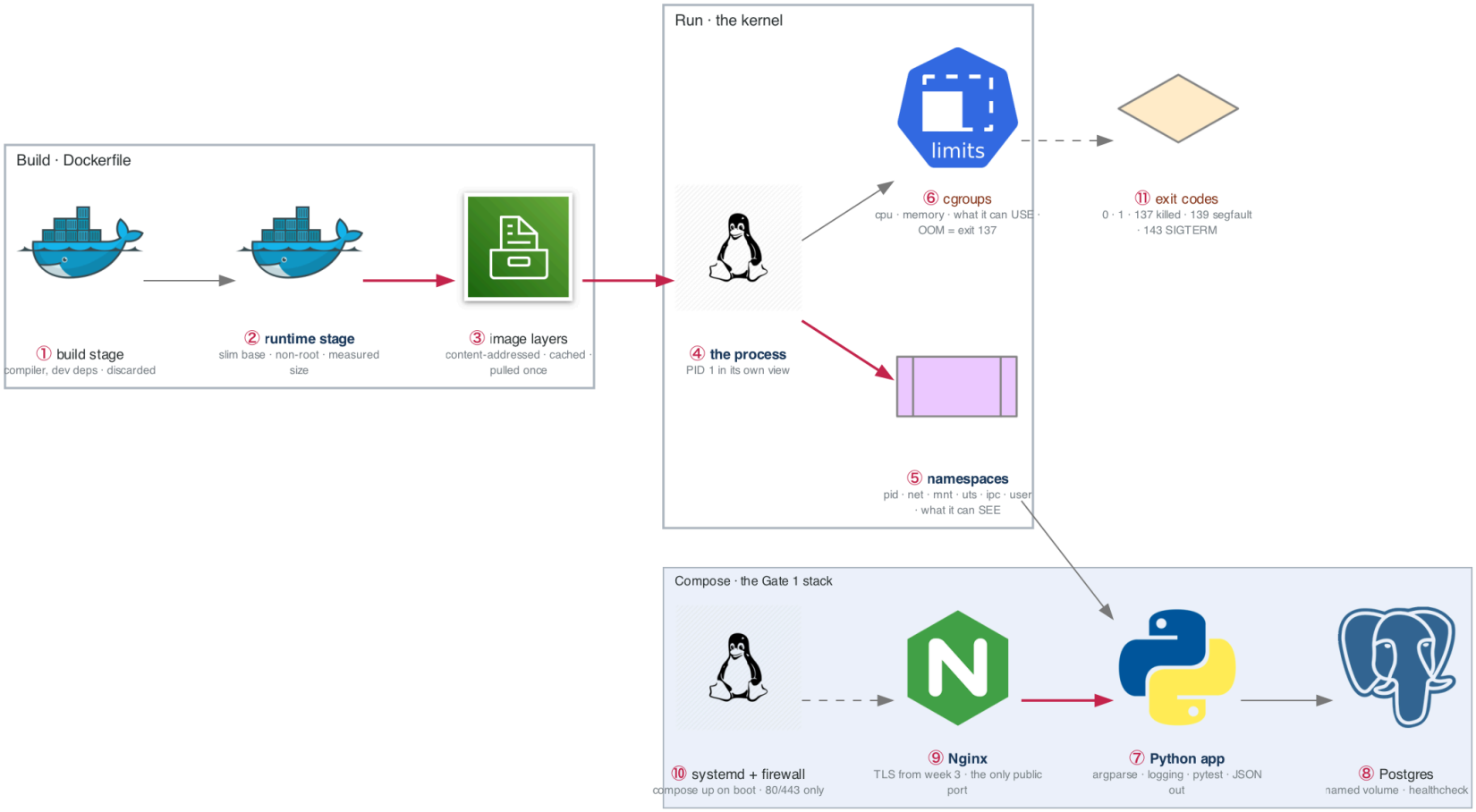
What a container actually is, and the stack Gate 1 asks you to deploy, numbered

Week 4 · Figure 4.1 · figure with badges 1–11 · 2026-10-17



What to say, badge by badge

About three and a half minutes at a normal pace (397 words). Point at each badge as you say its line.

Open
“A container is not a small VM. It is a Linux process with restricted visibility and a budget. This figure shows the build, the isolation, and the three-container stack Gate 1 asks you to deploy from nothing.”

1 build stage
“The first stage has the compiler and every dev dependency. Nothing from it ships except the artifact you copy out. This is why multi-stage builds shrink images by hundreds of megabytes.”

2 runtime stage
“A slim base, a non-root user, only the artifact. Measure before and after and put the two numbers in the README. Size is attack surface, and a number is proof.”

3 image layers
“Each instruction is a layer, addressed by content hash. Unchanged layers are cached and pulled once. Order the Dockerfile so the things that change most come last.”

4 the process
“`docker run` starts one process. Inside, it is PID 1. From the host, `ps` shows it as an ordinary process with a strange parent. There is no hypervisor anywhere in this picture.”

5 namespaces
“Six namespaces decide what the process can see: its own PIDs, its own network stack, its own mounts, hostname, IPC, and user mapping. `nsenter` lets you step into each one and prove it.”

6 cgroups
“cgroups decide what it can use. CPU over the limit is throttled; memory over the limit is killed, exit code 137. Week 15 shows the same rule in Kubernetes. Learn it here where it is cheap.”

7 Python app
“The week's first two days: a CLI with argparse, type hints, logging to stderr, JSON to stdout, and tests. It also exposes a Prometheus-format metrics endpoint, a preview of week 20.”

8 Postgres
“A named volume so data survives the container. A healthcheck so Compose knows when it is actually ready, not just started. Depends-on without a healthcheck is a race.”

9 Nginx
“The week-3 proxy, now in a container, still the only thing on a public port. TLS terminates here exactly as before.”

10 systemd and firewall
“A unit runs `compose up` on boot and restarts it on failure. The firewall allows 80 and 443 and nothing else. The gate checks both.”

11 exit codes
“0 is success, 1 is the app's own failure, 137 is the kernel killing it, 139 is a segfault, 143 is a clean SIGTERM. Reading the code is the first diagnostic step in BF-19.”

Close
“Build small, run isolated, budget enforced, three containers behind one proxy, managed by systemd. Then the Gate 1 question: narrate the packet from the browser to the Python process, box by box.”